Ridge Regression on Diamonds Dataset

A Comparative Analysis of Optimization Algorithms

Davide Gamba
Matr. 1053470

University of Bergamo
Department of Management, Information and Production Engineering

March 30, 2026

Introduction

The objective of this study is to conduct a comparative analysis of the convergence behavior of various optimization algorithms applied on a Ridge Regression problem to the Diamonds Dataset.

In particular, the focus is on:

- Evaluating the respective algorithms convergence rates.
- Investigating how theoretical properties of the objective function (such as Convexity, Smoothness ecc..) have an impact to results.
- Analyzing how the L_2 regularization parameter (λ) of Ridge Regression influences algorithms behavior.

The following algorithms were implemented:

- Gradient Descent (Naive, Bounded, Smooth).
- Accelerated Gradient Descent (Nesterov's).
- Mini-Batch Stochastic Gradient Descent (Decaying Learning rate)
- Newton's Method.
- Adagrad.

Dataset Overview: The Diamonds Dataset

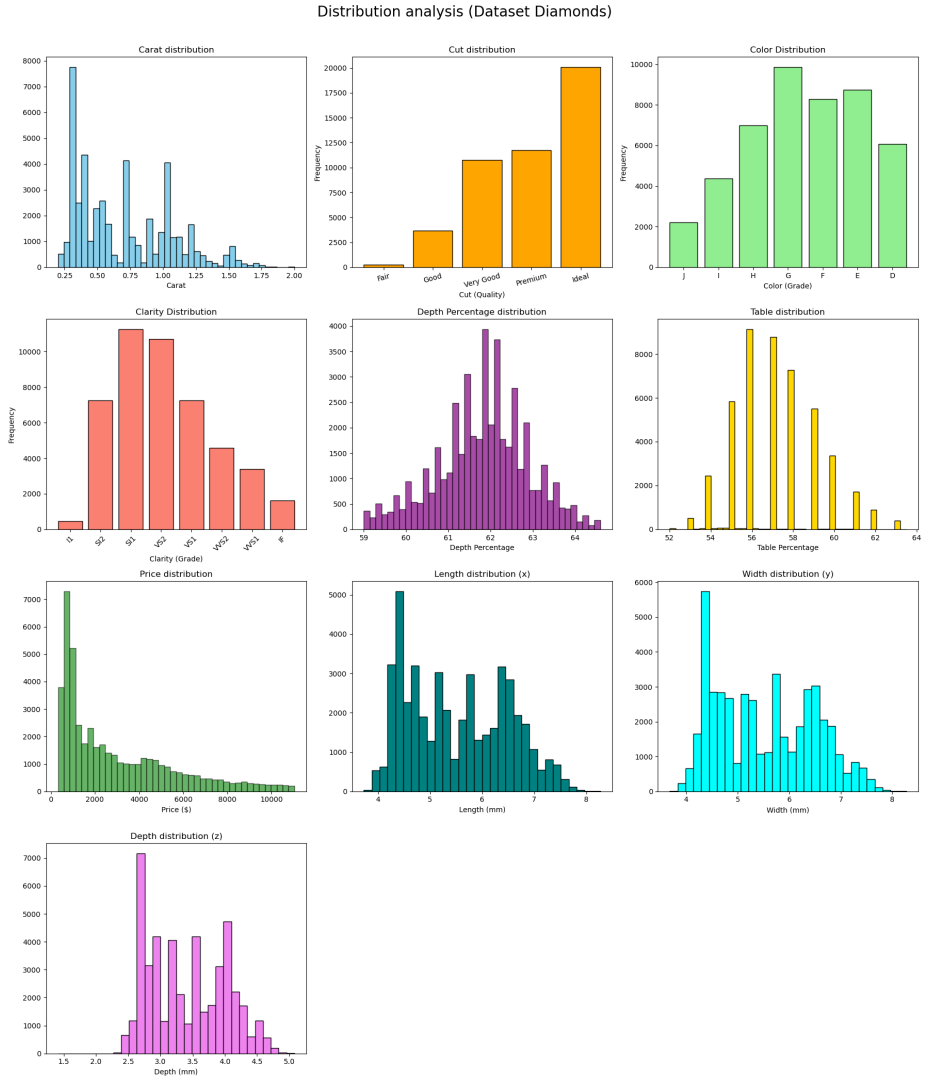
We utilized the **Diamonds Dataset**. The dataset consists of approximately **54,000 observations** ($n = 53.940$), each representing a diamond with **10 distinct attributes**. The goal is to predict with a Ridge Regression the **Price** of a diamond based on its physical and quality characteristics.

Feature Description:

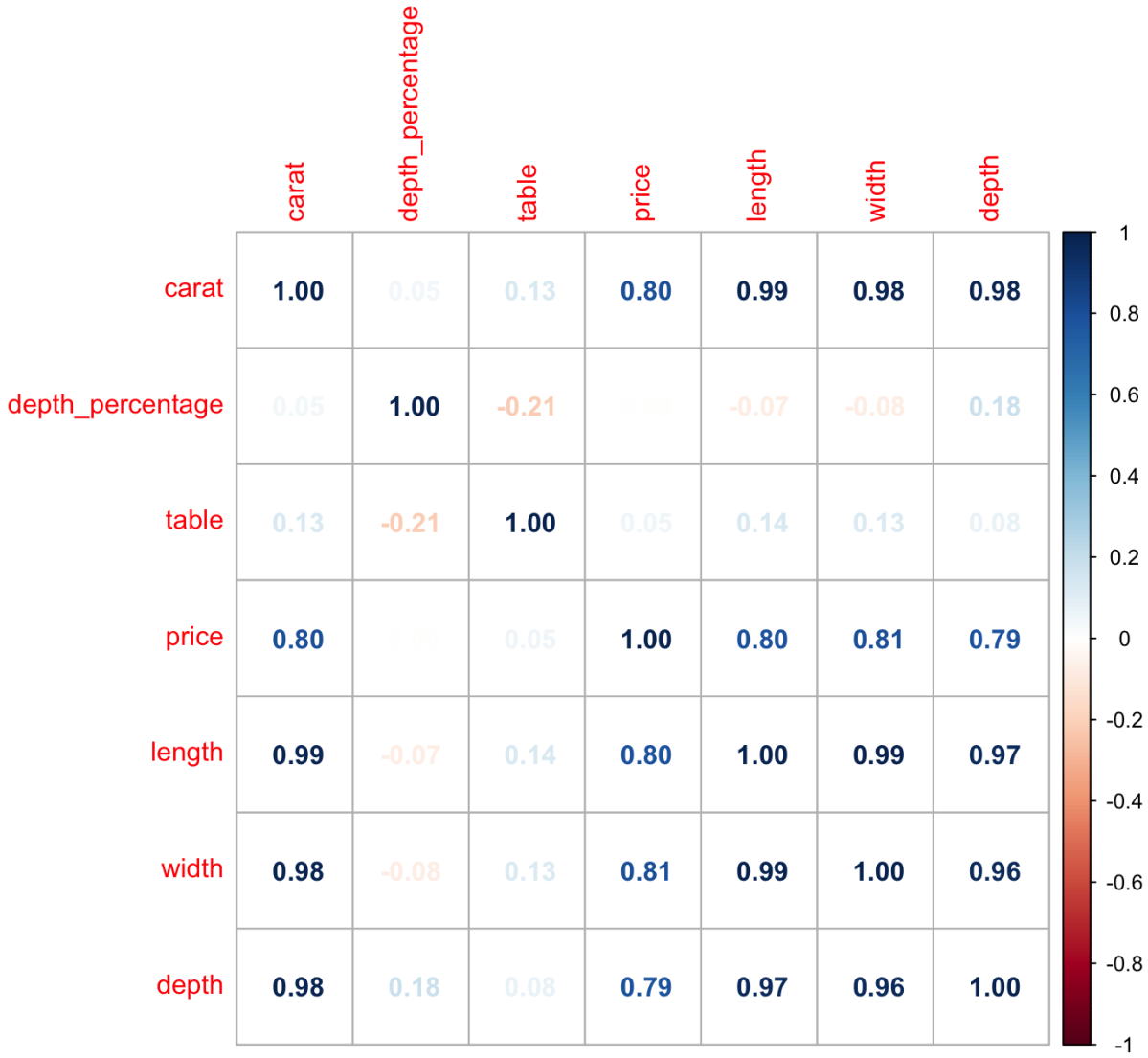
- **Target Variable (y):** `price` in US dollars.
- **Numerical Features:**
  - `carat`: Weight of the diamond.
  - `x`, `y`, `z`: Length, Width, and Depth in mm.
  - `depth`: The depth of the diamond divided by the diameter of the diamond.
  - `table`: The length of the table facet divided by the diameter of the diamond.
- **Categorical Features:**
  - `cut`: Quality of the cut (Fair, Good, Very Good, Premium, Ideal).
  - `color`: Diamond color, from J (coloured, worst) to D (colorless, best).
  - `clarity`: How clear the material is and what surface imperfections it have.

Dataset Overview: The Diamonds Dataset

Distribution of all the main variables within the diamonds dataset.



Dataset Characteristics: Feature matrix



Dataset Characteristics: Feature correlation

Multicollinearity:
In our dataset, there is a strong correlation between all variables, in particular between `carat` (weight) and the physical dimensions (`x`, `y`, `z`).

Impact on Optimization:

- Given the high multicollinearity among the features in the matrix X, we expect small eigenvalues for the Normal Matrix $X^T X$ and a loss objective landscape with flat and elongated valleys.
- Consequently, the optimization problem is **ill-conditioned** and particularly challenging for gradient-based methods.
- We will demonstrate that we can mitigate this issue by using Ridge Regression instead of standard linear regression.

Ridge Regression

Ridge Regression problem extends Ordinary Least Squares (OLS) by imposing an L_2 penalty on the model parameters to mitigate overfitting and multicollinearity.

The objective function is the mean squared errors for each data point ($i = 1$ to n) plus the sum of squared weights (from $j = 1$ to d).

$$f(w) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - x_i^T w)^2 + \frac{\lambda}{2} \sum_{j=1}^{d} w_j^2$$

We can define it in vector notation as follows:

$$f(w) = \underbrace{\frac{1}{2n} \|y - Xw\|^2}_{\text{Half MSE}} + \underbrace{\frac{\lambda}{2} \|w\|^2}_{\text{Regularization}}$$

Where:

- $X \in \mathbb{R}^{n \times d}$ is the feature matrix (includes a column of 1).
- $y \in \mathbb{R}^n$ is the target variable (price).
- $w \in \mathbb{R}^d$ is the weights vector.
- $\lambda > 0$ is the regularization hyperparameter.

Gradient and Hessian Matrix

Since $f(w)$ is differentiable everywhere, we can compute the first-order derivative (the Gradient) and second-order derivative (the Hessian matrix).

We will use them later in optimization's algorithm and to investigate the property of $f(w)$.

The Gradient $\nabla f(w)$

$$\nabla f(w) = \frac{1}{n} X^T (Xw - y) + \lambda w$$

The Hessian Matrix $\nabla^2 f(w)$

$$H = \nabla^2 f(w) = \frac{1}{n} X^T X + \lambda I$$

$f(w)$ is strictly convex

To investigate the convexity of our objective function, we rely on the Second-order Characterization of Convexity:

- *If $\nabla^2 f(\mathbf{x})$ exists at every point $\mathbf{x} \in \mathrm{dom}(f)$ and is symmetric. Then f is convex if and only if $\mathrm{dom}(f)$ is convex, and for all $\mathbf{x} \in \mathrm{dom}(f)$, we have $\nabla^2 f(\mathbf{x}) \succeq 0$. So $\nabla^2 f(\mathbf{x})$ is* **positive semidefinite**.
- *A symmetric matrix M is Positive Semi-Definite if $v^T Mv \geq 0$ for all vectors v, and Positive Definite if $v^T Mv > 0$ for all non-zero vectors $v \neq 0$.*

Let's apply this definition to our Ridge Regression Hessian matrix, analyzing its quadratic form $v^T Hv$ for any arbitrary non-zero vector $v \in \mathbb{R}^d$:

$$v^T Hv = v^T \left(\frac{1}{n} X^T X + \lambda I \right) v$$

$f(w)$ is strictly convex

Solving the quadratic form, we get:

$$v^T H v = \underbrace{\frac{1}{n}\|Xv\|^2}_{\text{OLS Base}} + \underbrace{\lambda\|v\|^2}_{\text{Ridge Penalty}}$$

- **The OLS Base:** The squared norm $\|Xv\|^2$ is a squared length, meaning it is always non-negative (≥ 0).
- **The Ridge Penalty:** For any non-zero vector $v \neq 0$, its squared norm $\|v\|^2$ is strictly positive. Since our regularization hyperparameter is strictly positive ($\lambda > 0$), the entire term $\lambda\|v\|^2$ is strictly greater than zero (> 0).

Because we are adding a strictly positive term to a non-negative term, the entire sum is guaranteed to be strictly positive.

Adding the ridge penalty guarentee us that Hessian matrix H is **Positive Definite** and consequently the objective function possesses a single unique global minimum, so that means $f(w)$ is strictly convex.

Ridge Regression - Properties

To understand how optimization algorithms will behave on Ridge Regression objective function $f(w)$, we must analyze its properties.

1. Convexity: Yes (Strictly Convex)
As proven in the previous section, the addition of the L_2 penalty ensures that the Hessian matrix H is Positive Definite. Therefore, the function is not just convex, but **strictly convex**. This guarantees that any local minimum is the unique global minimum w^*.

2. Lipschitz Continuous: No (Globally)
A function $f(w)$ is globally Lipschitz continuous if its gradients are bounded in norm everywhere ($\|\nabla f(w)\| \leq B$). Our objective function $f(w)$ is a quadratic function. As the weights w can grow toward infinity, the gradient $\nabla f(w) = \frac{1}{n} X^T(Xw - y) + \lambda w$ also can grows linearly toward infinity. Because the gradient is unbounded over the entire domain $\mathbb{R}^d$, the function itself is **not globally Lipschitz continuous**.

Ridge Regression - Properties

3. Smoothness (L-smooth): Yes
It is L-**Smooth**. The curvature does not change abruptly and the gradient doesn't oscillate wildy, the function always lies *below* a certain parabola:

$$f(y) \leq f(x) + \nabla f(x)^T (y - x) + \frac{L}{2}\|x - y\|^2 \quad \forall x, y$$

- **Value of L:** It corresponds to the largest eigenvalue of the Hessian matrix (the maximum curvature of the loss landscape).

$$L = \lambda_{\max}\left(\frac{1}{n} X^T X\right) + \lambda$$

- **Consequence:** This value acts as the absolute **speed limit** for Gradient Descent. The optimal safe step size is $\gamma = \frac{1}{L}$.

Ridge Regression - Properties

4. Strong Convexity (μ-strongly convex): Yes
It is μ-**Strongly Convex**. The curvature is "at least" μ in every direction. The function always lies *above* the quadratic function:

$$f(y) \geq f(x) + \nabla f(x)^T (y - x) + \frac{\mu}{2} \|x - y\|^2 \quad \forall x, y$$

- **Value of μ:** It corresponds to the smallest eigenvalue of the Hessian matrix.

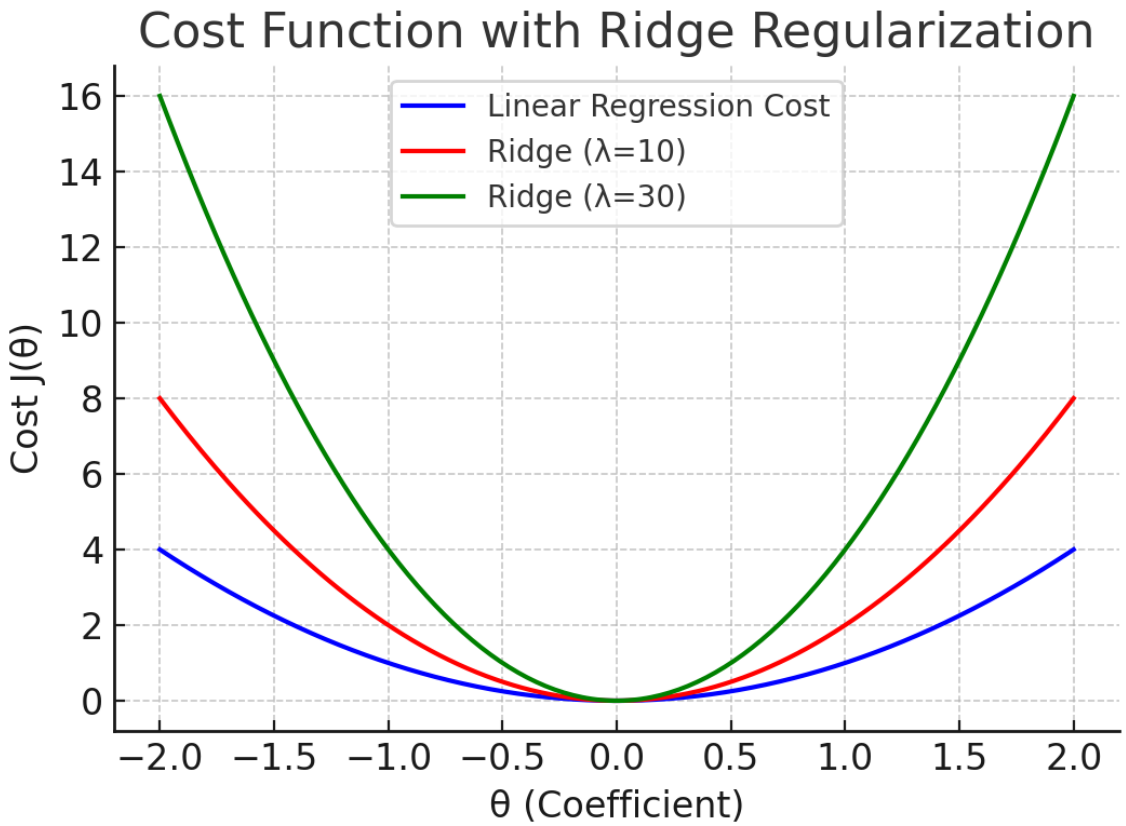
$$\mu = \lambda_{\min} \left(\frac{1}{n} X^T X \right) + \lambda$$

- **Consequence:** μ indicates how much the function is strongly convex.

The effect of $L2$ regularization

Specifically, increasing λ directly increases μ, the **strong convexity parameter**.

When we set a large λ, the minimum curvature (μ) is strictly bounded away from zero, the loss function becomes steeper and we eliminate flat regions where Gradient's based method might stall.



Dataset pre-processing steps

To prepare the dataset for our optimization algorithms, we now apply the following preprocessing steps:

- **Ordinal Encoding:** Categorical variables (`cut`, `color`, `clarity`) were mapped to integer values, preserving their inherent hierarchy.
- **Removing Outliers:** We filter out extreme anomalous data points.
- **Standardize the Features (X) and the Target (y):** By subtracting the mean and dividing by the standard deviation, we ensure all features and the target share the same scale (a mean of 0 and a variance of 1), also it allows Gradient Descent to point much more directly toward the minimum.
- **The Bias Trick (Adding a column of 1s):** We append a column of ones to our feature matrix X to absorb the intercept (or bias) term directly into our weight vector w.

Testing approach

To test the performance of the optimization algorithms, we use the following framework:

1. The Methodology:

- We create a predefined set of λ (regularization) values: $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}, 0.01, 0.1, 1.0\}$.
- We define five Ridge Regression problems, one fore each specific λ in the set, and we will apply every optimization algorithm to them.

2. The Goals:

By establishing these different regularization classes, we can evaluate:

- **Regularization Impact:** How varying regularization conditions affects the convergence speed and performance of the *same* algorithm.
- **Inter-Algorithm Comparison:** How *different* algorithms perform against each other when subjected to the *same* regularization conditions.

Testing approach

3. Early Stopping Mechanism:

- At each iteration, we evaluate if the algorithm has successfully reached the minimum of the loss function:

$$\epsilon = f(w_t) - f(w^*) \leq 10^{-6}$$

- If convergence is achieved, execution terminates immediately, returning the elapsed time and total iterations.
- Otherwise, the algorithm continues until it hits a predefined `max_iters` limit.

4. Exact Analytic Solution (Benchmark):

- We have to compute the exact optimal solution for every Ridge Regression subproblem to use it as a benchmark.
- We use the closed-form solution by `np.linalg.solve(A, b)`.

λ	10^{-5}	10^{-4}	10^{-3}	0.01	0.1	1.0
$f(w^*)$	0.04265771	0.04276069	0.04369175	0.04977553	0.07101217	0.16355421

Gradient Descent - Naive Learning Rate

Gradient descent by definition updates the weight vector w step-by-step by moving in the opposite direction of the gradient, scaled by the learning rate (step size) γ:

$$w^{(t+1)} = w^{(t)} - \gamma \nabla f(w^{(t)})$$

For this first run, we are using a **naive, very small learning rate $\gamma = 0.0001$** .

Choosing a very small step size is a conservative and highly stable approach: it guarantees that our algorithm will not overshoot the minimum or diverge.

But this safety comes with a heavy trade-off in efficiency: taking such microscopic steps means the algorithm might require a massive number of iterations to finally reach the optimal solution.

Gradient Descent - Naive Learning Rate

Final results per λ using a naive learning rate ($\gamma = 0.0001$):

λ	**Loss**	γ	**Iters**	**Time[s]**
10^{-5}	0.055102	0.0001	50000	9.55
10^{-4}	0.055118	0.0001	50000	10.53
10^{-3}	0.055281	0.0001	50000	9.70
0.01	0.056890	0.0001	50000	10.24
0.1	0.071706	0.0001	50000	11.27
1.0	0.163555	0.0001	31209	6.40

Observations:

- For low regularizations (from $\lambda = 10^{-5}$ to 0.1), the algorithm hits the `max_iters` limit of 50.000 without reaching the tolerance of the early stop.
- Only for the strongest regularization ($\lambda = 1.0$), the algorithm is pushed fast enough to converge in exactly 31.209 iterations.

Gradient Descent - Assuming Bounded Gradient

Assume the distance between initial solution and optimal solution is bounded by R and this bounded region contains all iterates.

Since we initialize our initial solution vector to **Zero** (`np.zeros`), the distance coincides exactly with the norm of the solution:

$$\|w_0 - w^*\| \leq R$$

$$\|0 - w^*\| = \|w^*\| \leq R$$

The assumption on R means: *"The optimal solution and the optimization path to reach it lie within a sphere of radius R."*

- Since we have **standardized the data** (X and y have zero mean and unit variance), the optimized weights w will naturally be small (typically between -1 and 1)
- We set $R = 5.0$. A generous safety margin (a "Bounded Region") that is virtually guaranteed to contain the true solution.

Gradient Descent - Assuming Bounded Gradient

We need to find a number B such that $\|\nabla f(w)\| \leq B$ for every w within the radius R. We rewrite the gradient formula in this way:

$$\nabla f(w) = \left(\frac{1}{n} X^T X + \lambda I\right) w - \frac{1}{n} X^T y$$

Using the triangle inequality ($\|a + (-b)\| \leq \|a\| + \|b\|$) and the definition of matrix norms:

$$\|\nabla f(w)\| \leq \underbrace{\left\|\frac{1}{n} X^T X + \lambda I\right\|}_{\text{This is smooth parameter L}} \cdot \underbrace{\|w\|}_{\leq R} + \underbrace{\left\|\frac{1}{n} X^T y\right\|}_{\text{Constant}}$$

So the formula for B is:

$$B = L \cdot R + \left\|\frac{1}{n} X^T y\right\|$$

Gradient Descent - Assuming Bounded Gradient

By making the assumption that our optimization path remains within a bounded region $\|w_0 - w^*\| \leq R$, and having calculated the maximum gradient bound B such that $\|\nabla f(w)\| \leq B$ for all w, we have effectively established that our objective function is **Lipschitz continuous** within this domain.

For Lipschitz continuous functions, optimization theory provides a specific, optimal step size: we can now set γ using the formula:

$$\gamma = \frac{R}{B\sqrt{T}}$$

where $T = 10.000$ iterations.

Gradient Descent - Assuming Bounded Gradient

Final results per λ using the bounded gradient step size (γ):

λ	**Loss**	γ	**Iters**	**Time[s]**
10^{-5}	0.049663	0.00214	10000	1.86
10^{-4}	0.049692	0.00214	10000	2.04
10^{-3}	0.049974	0.00214	10000	2.88
0.01	0.052629	0.00214	10000	2.15
0.1	0.071028	0.00210	10000	2.02
1.0	0.163555	0.00176	1768	0.32

Observations:

- **Larger Steps:** The dynamic $\gamma \approx 0.002$ is safely 20x larger than the naive guess.
- **Massive Speedup:** For $\lambda = 1.0$, it converges in only 1,768 iterations (vs. 31,209).
- **Higher Accuracy:** It reaches a strictly lower loss in just 10,000 steps compared to the 50,000 steps of the naive approach.

Gradient Descent - Smoothness

Previously we have seen that function is L-**Smooth**.

- **Value of L:** It is the largest eigenvalue of the Hessian matrix (the maximum curvature).

$$L = \lambda_{\max}\left(\frac{1}{n}X^T X\right) + \lambda$$

or equivalently

$$L = \left\|\frac{1}{n}X^T X\right\| + \lambda$$

- **Consequence:** This value is the **speed limit** for Gradient Descent.The optimal step size is $1/L$.

Gradient Descent - Smoothness

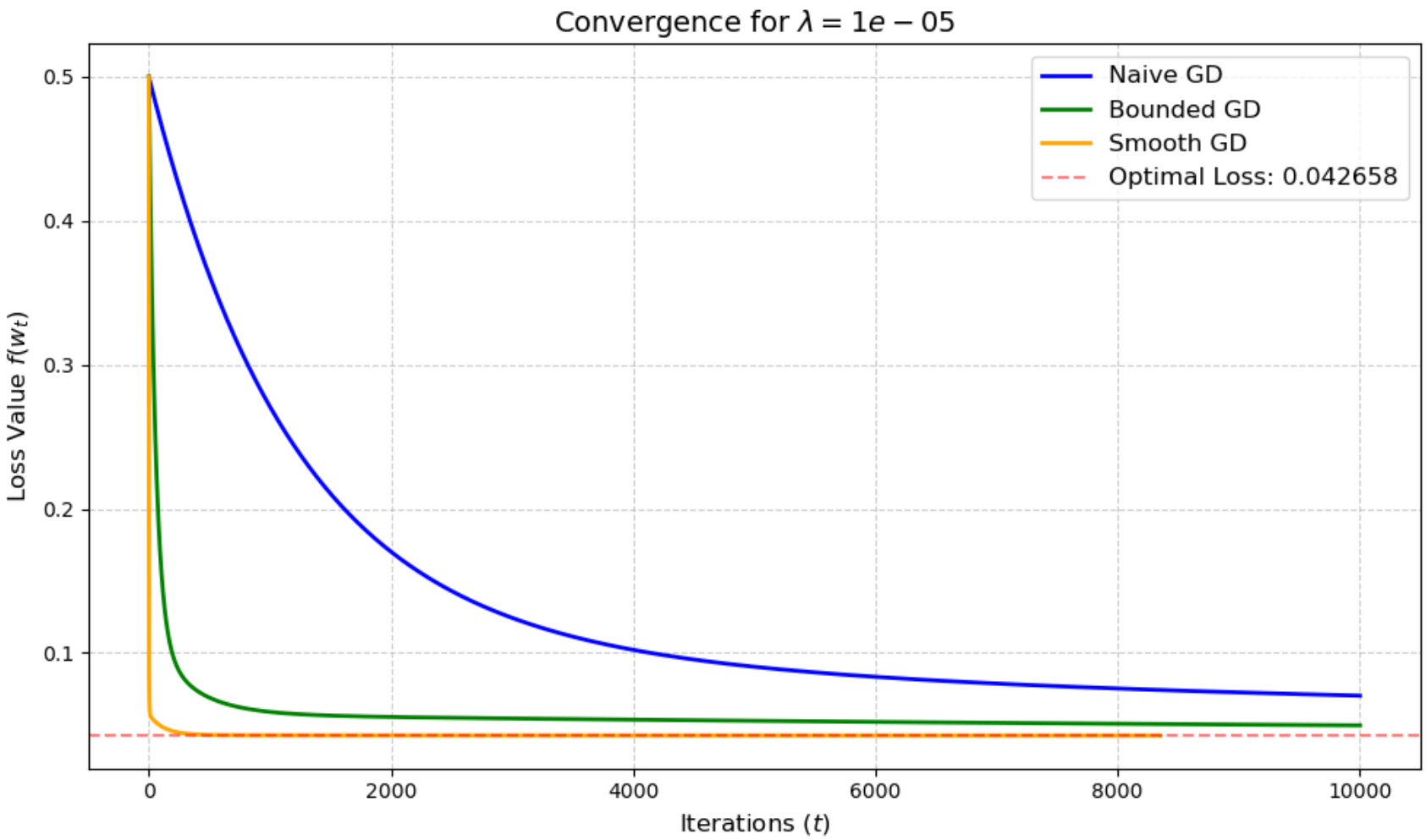
Final results per λ using the optimal step size ($\gamma = 1/L$):

λ	**Loss**	$\gamma = 1/L$	**Iters**	**Time[s]**
10^{-5}	0.042659	0.23247	8353	1.56
10^{-4}	0.042762	0.23247	7767	1.74
10^{-3}	0.043693	0.23242	4438	1.76
0.01	0.049777	0.23193	765	0.22
0.1	0.071013	0.22719	142	0.07
1.0	0.163555	0.18862	15	0.01

Observations:

- **Optimal Steps:** The safe learning rate ($\gamma \approx 0.23$) is $\sim$2,300x larger than our naive guess.
- **Lightning Fast:** For $\lambda = 1.0$, it perfectly converges in just 15 iterations (down from 31.209).
- **Perfect convergence:** For all case the algorithm converge to optimal solution.

Convergence Analysis: Comparing Gradient Descent implementations



Nesterov's Accelerated Gradient Descent

Now we implement the **Accelerated Gradient Descent** algorithm. This method improves standard gradient descent by computing the next iteration point as a dynamically weighted average between a conservative "smooth" step and a more "aggressive" step.

Choosing arbitrary initial points $\mathbf{z}_0 = \mathbf{u}_0 = \mathbf{w}_0$, for $t \geq 0$ we set:

$$\mathbf{u}_{t+1} := \mathbf{w}_t - \frac{1}{L}\nabla f(\mathbf{w}_t)$$

$$\mathbf{z}_{t+1} := \mathbf{z}_t - \frac{t+1}{2L}\nabla f(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} := \frac{t+1}{t+3}\mathbf{u}_{t+1} + \frac{2}{t+3}\mathbf{z}_{t+1}$$

- `w` represents the main weight vector.
- `u` represents the prudent/smooth update.
- `z` represents the aggressive update.

Nesterov's Accelerated Gradient Descent

Final results per λ using Accelerated Gradient Descent:

λ	**Loss**	**Iters**	**Time[s]**
10^{-5}	0.042659	203	0.10
10^{-4}	0.042761	202	0.12
10^{-3}	0.043692	152	0.07
0.01	0.049776	86	0.05
0.1	0.071012	23	0.01
1.0	0.163555	12	0.005

Observations:

- **Massive Speedup:** For ill-conditioned problem ($\lambda = 10^{-5}$), iterations go down from 8,353 to just 203 ($\sim$40x improvement).
- **Faster Convergence Rate:** Validates the theoretical improvement from $O(1/\epsilon)$ to $O(1/\sqrt{\epsilon})$.
- **Diminishing Returns:** For $\lambda = 1.0$ (a steep, well-conditioned bowl), the advantage is marginal (12 vs. 15 iterations).

Stochastic Gradient Descent with Mini-batches

Mini-batch Stochastic Gradient Descent (SGD) approximate the true gradient using a much smaller, randomly selected subset of the data of size m (where $m < n$).

Starting from an initial point $\mathbf{x}_0 \in \mathbb{R}^d$, at each iteration t we choose a random subset of indices $\{h_1, \ldots, h_m\} \subseteq \{1, \ldots, N\}$ and update the weights using a learning rate $\gamma_t \geq 0$:

$$\mathbf{w}_{t+1} := \mathbf{w}_t - \gamma_t \frac{1}{m} \sum_{j=1}^{m} \nabla f_{h_j}(\mathbf{w}_t)$$

- The algorithm operates in "epochs". At the start of an epoch, the entire dataset is randomly shuffled (`np.random.permutation`).
- Within an epoch, a loop sequentially iterates over this shuffled data and cut them in chunks of size `batch_size` $= 1000$ (our m).
- At each iteration the `minibatch_gradient` function calculates gradient on a small chunk, and the main loop updates the weights.

Mini batches SGD - Decaying Learning Rate

Since our objective function is μ-strongly convex. From **Tame Strong Convexity**, the learning rate to use is:

$$\gamma_t = \frac{2}{\mu(t+1)}$$

and it decays over each iteration.

However, applying this formula directly when our regularization parameter λ is small leads to immediate divergence.

- Due to multicollinearity of the dataset, the Hessian matrix of our objective function has very small eigenvalues.
- If μ is very small, the initial step size γ_t becomes too large, causing the algorithm to diverge.

Mini batches SGD - Decaying Learning Rate

We can demonstrate it: for our smallest ridge penalization ($\lambda = 10^{-5}$), the minimum curvature of our loss function is extremely flat, resulting in $\mu \approx 0.00130$. If we plug this value into the pure **Tame Strong Convexity** formula to calculate the learning rate for our very first iteration ($t = 0$), we get:

$$\gamma_0 = \frac{2}{0.00130} \approx 1538$$

This initial step size is catastrophically large and massively violates the theoretical speed limit $(2/L) \approx 0.46$ that prevents divergence for L-Smooth functions.

This same issue occurs across all other small values of the ridge regularization parameter λ.

Mini batches SGD - Decaying Learning Rate

To safely apply the optimal decay rate by the **Tame Strong Convexity**, we propose an adjusted learning rate:

$$\gamma_t = \frac{2}{L + \mu(t+1)}$$

In this way we get:

- **Initial Stability ($t = 0$):** At the very first iteration, the learning rate evaluates to exactly $\frac{2}{L+\mu} < \frac{2}{L}$. This respect the theoretical speed limit for smooth strongly convex functions and prevents the algorithm from diverging, regardless of how small μ is.

- **Asymptotic Convergence ($t \to \infty$):** As the iterations progress, the L term becomes dominated by μt. The learning rate transitions to approximate $\frac{2}{\mu t}$, like in the original Tame Strong Convexity formula.

Mini batches SGD - Decaying Learning Rate

Final results per λ using the adjusted decaying learning rate (γ_t):

λ	**Loss**	**Iters**	**Time[s]**
10^{-5}	0.042659	14199	3.58
10^{-4}	0.042761	15115	3.94
10^{-3}	0.043693	8104	2.23
0.01	0.049776	1730	0.71
0.1	0.071013	404	0.13
1.0	0.163555	101	0.04

Observations:

- **Guaranteed Convergence:** Safely bounded initial steps eliminate chaotic bouncing, achieving perfect convergence to the exact global minimum for every λ, but the stochastic noise makes Mini-Batch SGD slower compared to Smooth-GD.

Newton's Method

Until now, we have relied on first-order optimization algorithms (like Gradient Descent and its variants), which only use the gradient (the slope) to update weights.

Newton's Method introduces a massive upgrade by incorporating second-order derivative information—specifically, the curvature of the loss landscape represented by the Hessian matrix $\nabla^2 f(\mathbf{x})$.

Mathematically, for minimizing a convex function $f : \mathbb{R}^d \to \mathbb{R}$ in the d-dimensional case, the update rule is defined as:

$$\mathbf{w}_{t+1} := \mathbf{w}_t - \nabla^2 f(\mathbf{w}_t)^{-1} \nabla f(\mathbf{w}_t)$$

Newton's Method

Final results per λ using Newton's Method:

λ	**Loss**	**Iters**	**Time[s]**
10^{-5}	0.042658	1	0.006
10^{-4}	0.042761	1	0.004
10^{-3}	0.043692	1	0.003
0.01	0.049776	1	0.003
0.1	0.071012	1	0.003
1.0	0.163554	1	0.002

Observations:

- **Single update iteration:** Newton's Method achieves exact numerical convergence (to our 10^{-6} tolerance) in a single update step.

Newton's Method

This result is a direct consequence of the underlying mathematical nature of **Ridge Regression**. The objective function is a quadratic and strictly convex function with respect to the weight vector w.

$$f(w) = \frac{1}{2n}\|Xw - y\|^2 + \frac{\lambda}{2}\|w\|^2$$

The One-Step Convergence Lemma:
On nondegenerate quadratic functions, with any arbitrary starting point $w_0 \in \mathbb{R}^d$, Newton's method yields the global minimum w^* in exactly one step ($w_1 = w^*$).

Because the Ridge Regression objective function satisfies the condition of a nondegenerate quadratic function (a matrix with strictly positive eigenvalues has a non-zero determinant and is therefore invertible), this experimental behavior aligns exactly with optimization theory.

Adagrad

Adagrad modifies standard Stochastic Gradient Descent by introducing a dynamic, per-coordinate learning rate. At each iteration t, we first pick a stochastic gradient $\mathbf{g}_t$.

1. Gradient Accumulation
We accumulate the history of squared gradients for each individual coordinate i:

$$[G_t]_i := \sum_{s=0}^{t} ([\mathbf{g}_s]_i)^2 \quad \forall i$$

(This step tracks how strong the gradients have been in the past for the i-th coordinate).

2. Parameter Update
We then update each coordinate i of our parameter vector $\mathbf{w}$:

$$[\mathbf{w}_{t+1}]_i := [\mathbf{w}_t]_i - \frac{\gamma}{\sqrt{[G_t]_i}} [\mathbf{g}_t]_i \quad \forall i$$

Adagrad

The Adaptive Learning Rate:
The crucial part of this formula is the term $\frac{\gamma}{\sqrt{[G_t]_i}}$, which acts as our new coordinate-specific learning rate.

- By dividing the base learning rate γ by the square root of the accumulated gradients, the algorithm automatically **reduces the step size** for coordinates that have historically had large gradients.
- Instead, if a coordinate has had consistently small gradients, its learning rate remains relatively **higher**.

This mechanism speeds up convergence and reduces oscillations along directions where gradients are already large.

Adagrad

Final results per λ using the Adagrad algorithm with $\gamma = 0.1$:

λ	**Loss**	**Iters**	**Time[s]**
10^{-5}	0.042659	42511	11.99
10^{-4}	0.042762	37110	9.61
10^{-3}	0.043693	19376	4.73
0.01	0.049776	3477	0.78
0.1	0.071013	9059	1.98
1.0	0.163554	8292	1.92

Observations:

- **Guaranteed Convergence:** Adagrad successfully reached the exact global minimum for all λ values.
- **Strong γ Decay:** The accumulation term G_t drives the learning rate to zero very quickly, increasing the number of iterations required to achieve convergence.

Summary of Results for $\lambda = 10^{-5}$

Performance comparison on the ill-conditioned problem ($\lambda = 10^{-5}$):

Optimization Method	**Loss**	**Iterations**	**Time[s]**
Naive Gradient Descent	0.055102	50000*	∼10.50
Bounded Gradient Descent	0.049663	10000*	1.86
Smooth Gradient Descent	**0.042659**	8353	1.56
Accelerated Gradient Descent	**0.042659**	203	0.10
Mini-Batch SGD	**0.042659**	14199	3.58
Newton's Method	**0.042658**	1	0.006
Adagrad	**0.042659**	42511	11.99

**Hit maximum iterations limit without converging*

Summary of Results for $\lambda = 1.0$

Performance comparison on the strongly regularized problem ($\lambda = 1.0$):

Optimization Method	**Loss**	**Iterations**	**Time[s]**
Naive Gradient Descent	**0.163555**	31209	6.39
Bounded Gradient Descent	**0.163555**	1768	0.32
Smooth Gradient Descent	**0.163555**	15	0.01
Accelerated Gradient Descent	**0.163555**	12	0.005
Mini-Batch SGD	**0.163555**	101	0.04
Newton's Method	**0.163554**	1	0.002
Adagrad	**0.163554**	8292	1.92

Conclusions

- **Impact of λ on Convergence:** Increasing the regularization hyperparameter λ makes the objective function more strongly convex and the optimization's problem becomes better conditioned. Helps algorithms to converge and drastically reduces iterations and execution time.

- **Algorithm Performance Comparison:**
  - **Newton's Method:** Converges in exactly 1 step across all λ by leveraging second-order curvature.
  - **Accelerated GD:** Best first-order method, can effortlessly navigates flat valleys.
  - **Smooth GD ($1/L$):** Highly efficient for well-conditioned (high-λ) problems, but struggles heavily in flat scenarios.
  - **Mini-batch SGD:** Safely converges to the minimum, but its noisy updates make it less efficient than the deterministic Smooth-GD.
  - **Adagrad:** Auto-tunes step sizes without theoretical parameters, but the monotonically shrinking learning rate causes slow convergence.